SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION

IEEE 1016 Standard Format

Project: Smart Expense & Budget Management Application

Version: 1.1

Date: 02 October 2026

Status: Final

Group Number: G-15

Authors:
Rhea Sushanth – PES1UG24CS371
Sarthki Vachhani – PES1UG24CS422
Sai Manogna BV – PES1UG24CS399

The application is a layered Django monolith split into one app per business capability, with a service layer for business rules and an adapter isolating the optional sandbox bank service. This document follows the course Software Architecture and Design Specification template, with Part A (Section 3) covering architecture and Part B (Section 4) covering design.

Revision History

Version	Date	Author	Change Summary
1.0	02-10-2026	Group G-15	First complete draft: architecture, security, design and API sections
1.1	02-10-2026	Group G-15	Aligned with SRS v1.2 and Test Plan v1.1: component count and Alert Service interface corrected; admin defined as role-protected views; CSV duplicate handling, alert evaluation after import and login throttle clarified; budget, recurring and category endpoints completed; login, CSV import and class diagrams redrawn; references, figures and payload examples tidied.

Approvals

Role	Name	Signature/Date
Prepared by (Architecture, Security, UX design)	Sarthki Vachhani	Sarthki, 02-10-2026
Prepared by (Design, Data view)	Rhea Sushanth	Rhea, 02-10-2026
Prepared by (Component diagram, Import/Sandbox design, API design review)	Sai Manogna BV	Manogna, 02-10-2026

1. Introduction

This document describes how the G-15 system is structured (Part A, Architecture) and how its main behaviors, interfaces and errors are designed (Part B, Design), so that implementation can proceed directly from it.

1.1 Purpose

This document specifies the architecture and design of the Smart Expense & Budget Management Application, Group G-15, so that implementation can proceed directly from it and so that every architectural and design decision can be traced back to a requirement in the SRS.

1.2 Scope

It covers the single-user web application defined in the SRS: authentication, transactions and CSV import, budgets and alerts, reports, recurring transactions, admin category management, and the optional sandbox bank adapter. It follows the IEEE 1016 software design description approach.

1.3 Audience

Developers implementing the Django apps, QA engineers writing the test cases in the companion Test Plan, the course instructor and evaluators assessing IEEE-format compliance, and any future maintainer extending the system beyond this release.

1.4 Definitions

Term	Meaning
MVT	Model-View-Template, Django's variant of MVC
Service layer	Plain Python modules holding business rules, called by views
Adapter	A class that hides an external service behind a fixed interface
HTMX	Library for partial page updates using HTML fragments over HTTP
DTO	Data transfer object: the simple data structure passed between layers

2. Document Overview

2.1 How to use this document

Part A (Section 3) describes the system's architecture: its drivers, pattern, components, requirement traceability and security architecture. Part B (Section 4) describes the detailed design: sequence diagrams, API contracts, error handling, UX and open issues; the key classes, design decisions and data view supporting it are in Section 5. A reader who only needs the HTTP contracts can go directly to Section 4.3; a reader checking security controls can go directly to Section 3.9. All diagrams are drawn in standard UML notation (component, sequence and class diagrams).

2.2 Related Documents

- G-15 Software Requirements Specification v1.2 (Deliverable 2) and Synopsis (Deliverable 1).
- G-15 Test Plan v1.1 (companion document, IEEE 829 format).
- IEEE Std 1016-2009, Software Design Descriptions; IEEE Std 42010, Architecture Description.
- Django documentation (MVT, ORM, authentication, CSRF); OWASP Top 10 (2021).

Part A: Architecture

3. Architecture

3.1 Goals & Constraints

The pattern and component choices in this document are driven by the SRS constraints and the quality attributes below.

Driver	Source	Effect on the architecture
Dashboard and report responses within 2 s for 95% of requests	NFR-001	Aggregation queries in a dedicated `reports/queries.py`; database indexes on user, category and date
Business logic in separate, unit-tested modules	NFR-005	One Django app per capability; rules live in a service layer, not in views
Core services testable without the bank sandbox	NFR-006	Sandbox behind an adapter interface with a mock implementation
Re-import must not create duplicates	NFR-004	Import runs as an atomic, idempotent operation with a duplicate key check
Passwords hashed, RBAC, input sanitization, logging	SEC-001 to SEC-006	Security layer described in Section 3.9
Web only, single user, single currency, no real bank linking	SRS 2.5	Server-rendered monolith; no microservices, no external payments
Small team, 12-week Agile schedule	SRS Appendix E	Reuse Django built-ins; keep one deployable unit

3.2 Stakeholders & Concerns

Stakeholder	Concern
End user (Customer role)	Correctness of balances and alerts; financial data stays private (SRS 5.1.2)
Administrator role	Reliable category management; admin functions are restricted to staff
Development team (Group G-15)	Maintainability and testability (NFR-005, NFR-006); clear per-member module ownership (Synopsis)
Course instructor and evaluators	IEEE-format compliance, requirement traceability, demonstrable security controls
Future maintainers	Room to add real bank linking or multi-user budgets without a rewrite

3.3 Component (UML) Diagram

The diagram uses standard UML component notation: each box is a stereotyped component, the circles are provided interfaces (lollipop notation), dashed arrows are dependencies, and the database is shown as a separate node rather than inside the component boundary.

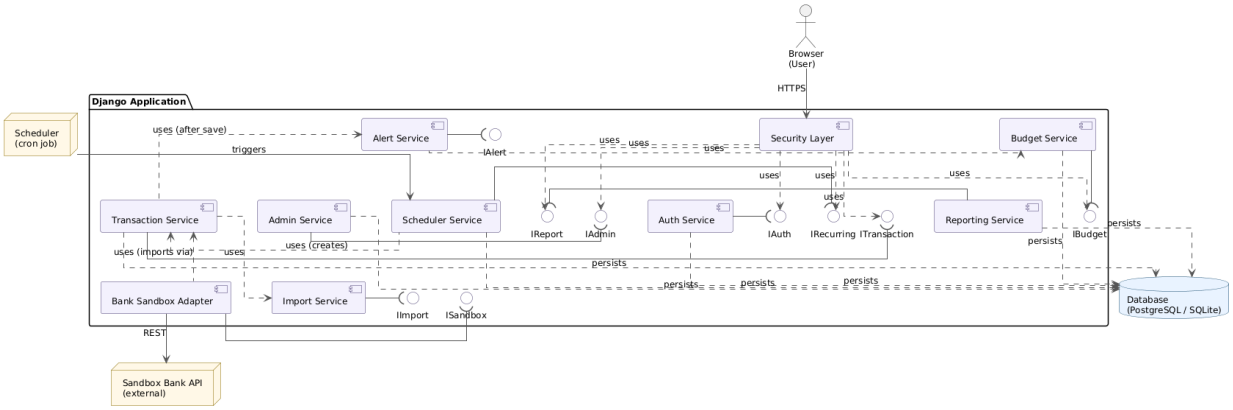


Figure 1: UML component diagram of the Django application, its 10 internal components, their provided interfaces, and the three external actors (browser, cron scheduler, sandbox bank API)

3.4 Component Descriptions

Component	Responsibility	Provided interface	Requirements
Security layer	Session check, CSRF check, role check, security event logging on every request	Django middleware and permission decorators	SEC-002, SEC-004, SEC-006
Auth Service (`accounts`)	Registration, login, logout, profile, password hashing and password policy	`register()`, `authenticate()`, `update_profile()`	FR-001 to FR-004, SEC-001
Transaction Service (`transactions`)	Create, edit, delete transactions; enforce one active category	`create_transaction()`, `update_transaction()`, `delete_transaction()`	FR-005 to FR-008
Import Service (`importer.py`)	Parse and validate CSV, skip duplicates (against the database and within the file), report bad rows, run atomically	`import_csv(file, user) -> ImportResult`	FR-009 to FR-011, NFR-004, SEC-003
Budget Service (`budgets`)	Create, edit, delete budgets; overlap check; cumulative spend calculation	`create_budget()`, `update_budget()`, `delete_budget()`, `spend_for(budget)`	FR-012 to FR-014
Alert Service (`alerts.py`)	Compare spend to threshold and limit for the budgets of the saved transaction's category whose period contains the transaction date; create alerts once per crossing	`evaluate(user, category, date)`	FR-015, FR-016
Reporting Service (`reports`)	Category and budget reports, trend and chart JSON	`category_report()`, `chart_data()`	FR-017, FR-018, NFR-001
Scheduler Service (`recurring`, `RuleRunner` class)	Store, edit and deactivate rules; on the run date create transactions and advance `next_run_date`	`create_rule()`, `update_rule()`, `run_due_rules(today)`	FR-019, FR-024
Admin Service (`categories`)	Add, edit, disable and re-enable categories; admin user list	Role-protected admin views (`categories/admin.py`)	FR-020 to FR-022

Component	Responsibility	Provided interface	Requirements
Bank Sandbox Adapter	Fetch transactions from the sandbox API behind a fixed interface, with a mock version	`fetch_transactions(user)`	FR-023, NFR-006, SEC-005
Database	Relational storage with foreign keys and constraints	Django ORM	SRS Appendix F

The ten application components are the Security layer, Auth, Transaction, Import, Budget, Alert, Reporting, Scheduler, Admin and Bank Sandbox Adapter; the Database is a separate node.

3.5 Chosen Architecture Pattern and Rationale

The system uses a layered client-server architecture on Django's MVT pattern, extended with a service layer and an adapter for external data sources. Each layer calls only the layer below it.

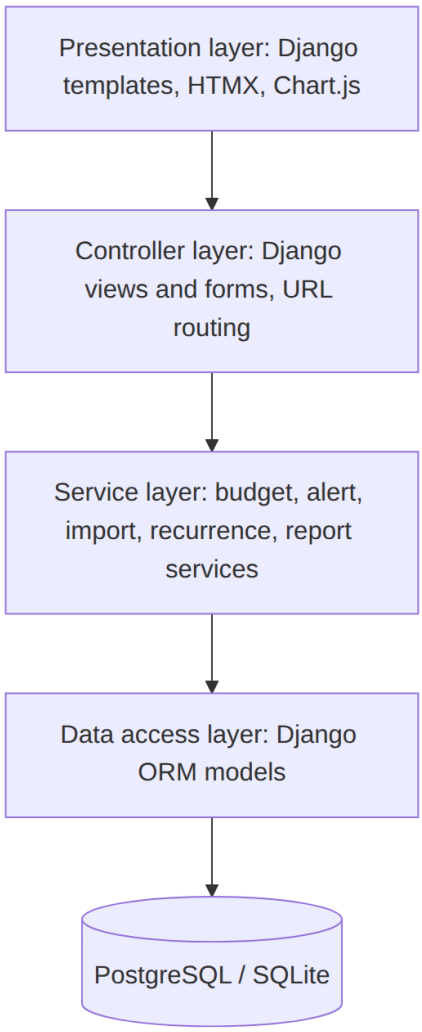


Figure 2: Layered architecture of the application

Layer	Responsibility	Main elements
Presentation	Renders pages, partial HTML fragments and chart data	Templates, HTMX attributes, Chart.js
Controller	Validates requests, checks permissions, calls services, returns a response	Views, forms, `urls.py`, decorators
Service	Holds all business rules so they can be unit tested without HTTP	`budgets/services.py`, `alerts.py`, `transactions/importer.py`, `recurring/tasks.py`, `reports/queries.py`
Data access	Persists entities and enforces integrity	Models for User, Category, Transaction, Budget, RecurringRule, Alert
Integration	Hides the optional bank sandbox	`SandboxAdapter` interface with a mock and a REST implementation

Alternative	Decision	Reason
Microservices	Rejected	Single-user app, 3-person team and 12 weeks; the network and deployment overhead adds no benefit
Plain Django with logic in views	Rejected	Breaks NFR-005 and NFR-006; business rules could not be unit tested without HTTP
Layered MVT with service layer	Chosen	Matches the Django tools the team reuses, keeps rules testable and gives one deployable unit

Two supporting patterns are used: the **Adapter pattern** isolates the sandbox bank service (FR-023, NFR-006), and an **Observer-style hook** raises alert evaluation after a transaction is saved, so the alert logic is decoupled from the entry views.

3.6 Technology Stack & Data Stores

Layer	Technology	Notes
Language and framework	Python 3, Django (MVT)	SRS Appendix E.5 tool stack
Frontend	Django templates, HTMX, Chart.js	Server-rendered pages with partial-fragment updates; no separate SPA front end
Database (development/test)	SQLite	Fast local runs and CI unit/integration tests
Database (staging/production)	PostgreSQL	Used for performance, system and security testing
Authentication	Django sessions, PBKDF2/Argon2 password hashing	No token-based auth; see Section 3.9
Background scheduling	OS/platform cron job calling `manage.py run_recurring`	Daily trigger for the Scheduler Service
Testing and quality	Pytest, Django test client, Coverage.py, Locust, OWASP ZAP	NFR-001, NFR-005, NFR-006, Section 3.9
CI/CD	GitHub Actions	Runs the full suite on every push and pull request
External integration	Sandbox bank REST API, behind the Bank Sandbox Adapter	Mocked for all testing except the optional live pass (FR-023)

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Single Django monolith becomes a bottleneck if usage grows beyond the single-user scope	Degraded performance, harder scaling (NFR-001)	Service layer keeps business logic isolated per app, so a module could be extracted into a separate service later without a rewrite
Daily cron job and the web process share one database without a dedicated job queue	Rare missed or duplicate recurring transaction	Scheduler Service processes each rule in its own transaction with an idempotent generation check (Sections 3.4, 4.2)
Sandbox bank API has no formal SLA and may be unavailable during grading	FR-023 cannot be demonstrated live	Adapter pattern isolates it; a mock adapter is used for all other testing and for the course demo (NFR-006)
Small three-person team owns ten components	Slower response to defects, bus factor	One Django app per capability with a named owner (Test Plan Section 9); layered design limits the blast radius of a change
Secrets (database credentials, any future bank keys) committed by accident	Credential leak	Environment variables only, secrets excluded from version control, dependency scanning in GitHub Actions (Section 3.9)

3.8 Traceability to Requirements

Every SRS requirement is allocated to at least one architectural component, and the table extends the SRS Section 8 matrix with the design sections that realize it.

Requirement(s)	Use case	Component	Design section
FR-001, FR-002, FR-004	UC-01	Auth Service	4.2 Login sequence, 4.3 Auth API
FR-003	UC-01	Auth Service	4.3 Auth API
FR-005, FR-006, FR-007	UC-02	Transaction Service	4.2 Add transaction sequence, 4.3 Transaction API
FR-008	UC-02	Transaction Service	4.3 Transaction API
FR-009, FR-010, FR-011	UC-05	Import Service	4.2 CSV import sequence, 4.3 Import API
FR-012, FR-013, FR-014	UC-03	Budget Service	4.3 Budget API
FR-015, FR-016	UC-07	Alert Service	4.2 Add transaction sequence
FR-017, FR-018	UC-04	Reporting Service	4.3 Report API
FR-019	UC-08	Scheduler Service	4.3 Recurring API
FR-024	UC-11	Scheduler Service, Scheduler	4.2 Recurring generation sequence
FR-020, FR-021, FR-022	UC-06, UC-09	Admin Service	4.3 Admin API
FR-023	UC-10	Bank Sandbox Adapter	4.3 Import API (sync endpoint)
NFR-001	UC-04	Reporting Service, database indexes	3.6, 5.6 data view
NFR-002	All	Deployment	Section 5.6 deployment view
NFR-003	UC-02	Presentation layer	Section 4.5 UX design; HTMX inline form flow (Section 4.3)
NFR-004	UC-05	Import Service	4.2, 4.4 Error handling
NFR-005	All	Layered structure, one app per capability	Section 3.5

Requirement(s)	Use case	Component	Design section
NFR-006	UC-03, UC-07	Service layer, Bank Sandbox Adapter mock	Sections 3.5, 3.4
NFR-007	All	Deployment	Section 5.6 deployment view
SEC-001 to SEC-006	UC-01, UC-05, UC-06, UC-10	Security layer and services	Section 3.9

3.9 Security Architecture

Security is enforced in layers, so no single missed check exposes user data: transport, request filtering, authorization in views, validation in services, and safe storage.

Security objectives

Objective	Architectural response
Confidentiality: a user sees only their own data	Every query is filtered by `request.user`; passwords are hashed; HTTPS only
Integrity: only the owner changes data; input cannot alter the system	Ownership checks in services, ORM parameterized queries, CSRF tokens, validated CSV input
Accountability and access control: admin functions restricted, events auditable	Role-based permission decorators, session expiry, security event log

Request path and controls

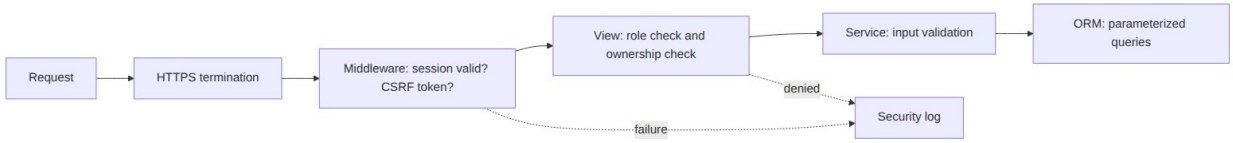


Figure 3: Request path and security controls

Controls mapped to requirements

SEC ID	Control	Where implemented
SEC-001	Django `PBKDF2PasswordHasher` (salted, iterated); Argon2 as an optional upgrade; plaintext never stored or logged	accounts using Django auth
SEC-002	`Role` field on User; `@role_required('admin')` decorator and mixin on every admin view, returning 403 to Customers; Django's built-in admin site is moved to `/django-admin/` and limited to superusers (developer use only)	`accounts/permissions.py`
SEC-003	Django forms and validators for every field; CSV size limit (2 MB), UTF-8 check, formula-prefix neutralization; template auto-escaping	`importer.py`, forms
SEC-004	Session cookies with `HttpOnly`, `Secure`, `SameSite=Lax`, 30-minute idle timeout; `login_required` on all non-public views	Settings and middleware
SEC-005	Adapter stores no bank tokens: it holds a token in memory for one sync call only and never writes it to the database or logs	`sandbox_adapter.py`
SEC-006	Logger `security` records failed logins, forbidden requests and permission denials with timestamp, user, IP and event type; logger `audit` records administrator category changes; passwords are never logged	Middleware and `accounts/`

Additional controls: a login attempt throttle (5 failures in 10 minutes locks the account for 10 minutes; attempts are counted per email address whether or not an account exists, so the 429 response does not reveal which emails are registered), `SECURE_SSL_REDIRECT`, HSTS, secrets in environment variables and never in the repository, and dependency scanning in GitHub Actions.

Threats considered (STRIDE)

Threat (STRIDE)	**Example**	**Mitigation**
Spoofing	Password guessing	Hashing, throttle, generic login error
Tampering	Editing another user's transaction by changing the ID	Owner-scoped queries returning 404
Repudiation	Denied admin change	Audit log of admin actions
Information disclosure	Stack trace or another user's data in an error page	`DEBUG=False`, custom error pages, owner filtering
Denial of service	Very large CSV upload	2 MB upload limit, timeouts
Elevation of privilege	Customer calls admin URL	Role decorator on every admin view